

Camunda 8 Incident Resolution Runbook

Practical troubleshooting guide for developers, workflow operators, and process owners

Scope: Camunda 8 / Zeebe / Operate • Focus: common process-execution incidents

Core rule Do not simply click Retry. First identify and correct the underlying cause, then retry/resolve the incident. If the cause remains, Camunda can create the incident again.

1. How Camunda 8 Incidents Work

An incident represents a problem that prevents a process instance from advancing. A common path is a job failure: the worker fails the job, retries are consumed, and an incident is created when no retries remain. Other incidents can arise from expression evaluation, decision evaluation, timers, uncaught BPMN errors, or other execution problems.

- Operate is the primary place to inspect process instances, incident messages, variables, and runtime state.
- A job-related incident generally requires fixing the worker/external dependency or correcting process data, restoring retries, and resolving/retrying the incident.
- A process-instance incident may require correcting variables or model/runtime data before retrying.
- A retry is a recovery action, not the root-cause fix.

2. Standard Incident-Resolution Workflow

1. Locate the affected process instance in Operate.
2. Open the incident and capture the process instance key, process definition/version, BPMN element ID, job type/job key (if applicable), error message, and current variables.
3. Classify the failure: worker/application, external dependency, variable/data, FEEL/expression, message correlation, BPMN error, timer, connector/secret, or platform/infrastructure.
4. Determine whether the failure is transient or deterministic. Transient failures usually justify retries; deterministic failures usually require data/model/application changes first.
5. Fix the underlying cause. Examples: restore a service, correct a variable, deploy a worker fix, correct a secret, or update the BPMN model.
6. If needed, update the process variables and/or job retries.
7. Resolve/retry the incident in Operate.
8. Verify that the process advances past the failed element and that no new incident is created.
9. Document the root cause and resolution if the incident is likely to recur.

Safety check before retrying Ask: “If this job runs again, could it duplicate an external side effect?” For payments, orders, emails, database writes, or API calls, confirm the worker is idempotent or that the external operation is safely repeatable.

3. Quick Triage Matrix

Symptom / message pattern	Likely cause	What to inspect	Typical recovery
---------------------------	--------------	-----------------	------------------

Job failed; retries = 0	Worker exception or external service failure	Incident message, job type, worker logs, dependency health	Fix cause → restore retries → resolve/retry
Cannot compare STRING and INTEGER / condition failed	Wrong variable type/value	Variables + FEEL expression	Correct variable → retry
Required variable missing / null	Bad input or mapping	Variables, input/output mappings	Add/correct variable → retry
FEEL expression evaluation failed	Invalid expression, null, wrong type	Expression + variables + model version	Fix data/expression → redeploy if model changed → retry
BPMN error not caught	Error code has no matching catch event	Worker error code + BPMN error boundary/event subprocess	Fix model/error code → redeploy for new instances; handle existing instance carefully
Message not correlated	No matching subscription/name/correlation key	Message name, correlation key, process state	Fix publisher/key/subscription → publish/correlate again
Timer not firing as expected	Expression/configuration/time-zone or instance state	Timer definition, incident/state, variables	Correct model/data → retry or allow timer to trigger
Connector/secret resolution failure	Missing/inaccessible secret or invalid configuration	Incident text, secret store, broker/connector logs	Fix secret/config → resolve/retry
Many instances fail at same task	Shared dependency or bad deployment	Failure timestamps, job type, deployment health	Fix shared cause → batch retry where appropriate

4. Common Incident Playbooks

4.1 Job Worker Failure / Retries Exhausted

Typical symptom: A service task is stuck with an incident after the worker repeatedly failed the job.

Diagnose

- Read the incident's error message; it should contain the worker/application failure context.
- Identify the job type and job key and inspect the worker logs for the same execution window.
- Check whether the failure is caused by bad process data, an unavailable dependency, an application defect, authentication, or a timeout.
- Check whether the external dependency is currently healthy before retrying.

Resolve

10. Fix the worker or external dependency.
11. If process data caused the failure, correct the relevant process variable.
12. Restore an appropriate number of job retries.
13. Resolve/retry the incident.
14. Confirm the job is activated and completes successfully.

Prevention

- Use meaningful failure messages in workers.
- Use retry backoff for transient external-system failures.
- Make workers idempotent because job handling follows an at-least-once execution model.

4.2 Wrong Variable Type / Invalid Variable Value

Typical symptom: An expression or gateway cannot evaluate because a variable has the wrong type, is null, or contains an unexpected value.

Diagnose

- Open Variables in Operate and compare the actual value/type with what the BPMN/FEEL expression expects.
- Inspect input mappings and the process that created the variable.
- Check whether the value came from a worker, API request, message, or previous task.

Resolve

15. Correct or add the variable with the expected type/value.
16. Verify dependent expressions or mappings are compatible with the corrected value.
17. Retry the affected process instance/incident.
18. Verify the process takes the intended path.

Prevention

- Validate external inputs before starting processes.
- Use consistent schemas and explicit variable mappings.
- Test FEEL expressions with representative null and boundary values.

4.3 FEEL / Condition Expression Failure

Typical symptom: A gateway, event, mapping, or other expression cannot be evaluated.

Diagnose

- Read the incident message to identify the expression and evaluation error.
- Check referenced variables for missing/null values and incorrect types.
- Inspect the deployed BPMN version to ensure the expression matches the current model.

Resolve

19. Correct the process data if the data is wrong.
20. If the BPMN expression is wrong, correct and redeploy the process definition.
21. For an existing instance, determine whether the instance can safely be retried against the corrected model/data.
22. Retry and verify the resulting path.

Prevention

- Keep expressions simple and test them with realistic data.
- Define expected variable types and validate inputs at process boundaries.

4.4 Missing Variable

Typical symptom: An expression, mapping, connector, or worker expects a variable that is absent.

Diagnose

- Check Variables at the incident's scope.
- Trace where the variable should have been created: start payload, message, previous task, mapping, or worker output.
- Check input/output mappings for incorrect variable names or scopes.

Resolve

23. Add/correct the missing variable if the instance data is recoverable.
24. If the model is responsible, correct the BPMN/mapping for future instances.
25. Retry the incident.
26. Verify downstream tasks receive the expected variable.

4.5 BPMN Error Is Not Caught

Typical symptom: A worker throws a BPMN error but the process produces an incident because no matching catch event handles it.

Diagnose

- Inspect the worker's error code.
- Find the corresponding boundary error event or error event subprocess in the BPMN model.
- Verify the error code matches exactly and that the catch event is attached to the correct scope.

Resolve

27. Correct the worker's BPMN error code or the BPMN model so the intended catch event matches.
28. Deploy the corrected process definition for future instances.
29. For an already-running instance, follow your organization's approved process-instance modification/recovery procedure rather than blindly retrying.

Prevention

- Use BPMN errors for expected business exceptions, not generic technical failures.
- Keep error codes stable and documented.

4.6 Message Not Correlated

Typical symptom: A message is published but the waiting process instance does not continue.

Diagnose

- Confirm the process instance is actually waiting at a message catch event.
- Compare the published message name with the subscription's message name.
- Compare the published correlation key with the subscription's correlation key.
- Check whether the publisher sent the expected value/type and whether the message was sent before the subscription existed.

Resolve

30. Correct the message name/correlation key or publisher logic.
31. If the message is intentionally correlated after the subscription is available, publish it again according to the application's delivery semantics.
32. Verify the subscription is consumed and the process advances.

Prevention

- Treat the message name and correlation key as a contract between publisher and process.
- Log correlation identifiers at the publishing boundary.

4.7 Timer / Scheduling Problem

Typical symptom: A process appears stuck waiting for a timer, or a timer-related expression causes an incident.

Diagnose

- Confirm the instance is waiting at the expected timer event.
- Inspect the timer definition and any variables used by its expression.
- Check for incorrect duration/date expressions or unexpected data.
- Check whether the process model/version is the one actually running.

Resolve

33. Correct the data or BPMN timer definition as appropriate.
34. If the issue is an expression/data incident, correct the cause and retry.
35. For a model change, deploy the corrected version and follow your organization's procedure for already-running instances.

4.8 Connector / Secret Resolution Failure

Typical symptom: A connector or job cannot resolve a required secret/configuration and an incident is raised.

Diagnose

- Read the incident message for secret-resolution details.
- Verify the secret exists and the runtime has permission to access it.
- Check the configured secret name/reference and relevant broker/connector logs.
- Determine whether the problem is missing secret, access denied, unavailable secret store, or invalid job-variable injection.

Resolve

36. Fix the secret or access/configuration problem.
37. Resolve/retry the incident.
38. Verify the connector/job activates and completes.

Prevention

- Manage secrets outside BPMN source code.
- Validate secret names and permissions during deployment testing.

4.9 Many Instances Fail for the Same Reason

Typical symptom: Multiple process instances have incidents at the same BPMN element/job type.

Diagnose

- Group incidents by process definition, version, BPMN element, job type, and time.
- Check whether a shared worker, API, database, credential, or deployment failed.
- Do not retry a large population until the shared root cause is confirmed fixed.

Resolve

39. Restore the shared dependency or deploy the corrected worker/model.
40. Test one representative instance first.
41. Use Operate's batch retry capability only after the fix is verified and the selected population is safe to retry.
42. Monitor the retry results for new incidents.

Prevention

- Alert on incident spikes rather than treating every instance independently.
- Record common failure signatures to speed up future triage.

5. Special Cases and Important Distinctions

Technical failure vs. business exception

A technical failure such as a temporary database outage is usually handled with job retries and, when necessary, an incident. An expected business exception such as “customer is not eligible” is generally better represented explicitly in the BPMN process using BPMN error handling or another modeled business path. Do not turn expected business outcomes into endless technical retries.

Retry vs. BPMN Error

- Retry: the same job should be attempted again because the work may succeed after a transient failure or after correcting data.
- BPMN Error: the process should follow an explicitly modeled exception path.
- Incident: the process cannot continue automatically and needs an operator/developer to correct the underlying problem.

Existing instance vs. future instances

Changing and redeploying a BPMN model primarily affects newly created instances unless you explicitly modify existing instances using supported runtime operations. When an incident involves an already-running process, distinguish between fixing the model for future executions and safely recovering the existing instance.

6. Worker Troubleshooting Checklist

- Is the worker process running?
- Is it connected to the correct Camunda cluster/environment?
- Is it subscribed to the exact job type used by the BPMN task?
- Are authentication credentials valid?
- Is the worker receiving jobs?
- Does the worker log the job key and relevant business correlation ID?
- Does the worker distinguish retryable technical errors from non-retryable business errors?
- Does it provide a useful error message when failing a job?
- Is retry backoff configured for transient dependencies?
- Could a retry duplicate an external side effect?
- Are worker and BPMN versions compatible?

7. Process-Data Troubleshooting Checklist

- Are required variables present?
- Are variable names exactly correct?
- Are values the expected types?
- Could a variable be null or empty?
- Are input/output mappings writing to the expected scope?
- Did a previous task actually produce the expected output?
- Did a message or API request supply the expected payload?
- Could stale data be causing a deterministic failure?

8. What to Capture Before Escalation

- Environment/cluster
- Process definition ID and version
- Process instance key
- BPMN element ID/name
- Incident key
- Job key and job type, if applicable
- Full incident error message
- Approximate failure timestamp
- Relevant process variables (excluding secrets/PII)
- Worker/application logs for the same execution window

- Recent deployments/configuration changes
- Status of dependent systems such as APIs, databases, queues, and secret stores
- Steps already attempted and their results

9. Recommended Incident Record

Field	Record
Incident	[incident key / link]
Process	[process definition ID + version]
Instance	[process instance key]
BPMN element	[element ID / name]
Job	[job type / job key, if applicable]
Symptom	[what the operator observed]
Root cause	[confirmed technical/data/model cause]
Fix	[what was changed]
Recovery	[retry / variable update / redeploy / other]
Verification	[how successful recovery was confirmed]
Prevention	[test, alert, code/model improvement]

10. Operator Do / Don't

Do	Don't
Read the incident message before acting.	Blindly retry every incident.
Fix the underlying cause first.	Treat retry as the root-cause fix.
Test one recovered instance before a bulk retry.	Bulk-retry a population while the dependency is still failing.
Check idempotency before retrying side effects.	Assume a worker runs exactly once.
Remove or redact secrets/PII from escalation notes.	Paste credentials, tokens, or sensitive payloads into tickets.
Separate technical failures from expected business exceptions.	Use technical retries for business decisions.

11. Reference Links

- **Camunda 8 — Incidents:** <https://docs.camunda.io/docs/components/concepts/incidents/>
- **Camunda 8 — Resolve incidents and update variables:**
<https://docs.camunda.io/docs/components/operate/userguide/resolve-incidents-update-variables/>
- **Camunda 8 — Job workers:** <https://docs.camunda.io/docs/components/concepts/job-workers/>
- **Camunda 8 — Dealing with problems and exceptions:**
<https://docs.camunda.io/docs/components/best-practices/development/dealing-with-problems-and-exceptions/>
- **Camunda 8 — Messages:** <https://docs.camunda.io/docs/8.7/components/concepts/messages/>
- **Camunda 8 — Secret resolution incidents:**
<https://docs.camunda.io/docs/next/components/concepts/secret-resolution-incidents/>

Version: September 2026 • Use this as an operational runbook and adapt steps to your organization's change-management and access-control procedures.